2024 IEEE International Conference on Robotics and Automation (ICRA)
May 13-17, 2024. Yokohama, Japan

FC-Planner: A Skeleton-guided Planning Framework for Fast Aerial Coverage of Complex 3D Scenes

Chen Feng[2], Haojia Li[2], Mingjie Zhang[1], Xinyi Chen[2], Boyu Zhou[1,†], and Shaojie Shen[2]

***Abstract*— 3D coverage path planning for UAVs is a crucial problem in diverse practical applications. However, existing methods have shown unsatisfactory system simplicity, computation efficiency, and path quality in large and complex scenes. To address these challenges, we propose FC-Planner, a skeleton-guided planning framework that can achieve fast aerial coverage of complex 3D scenes without pre-processing. We decompose the scene into several simple subspaces by a skeleton-based space decomposition (SSD). Additionally, the skeleton guides us to effortlessly determine free space. We utilize the skeleton to efficiently generate a minimal set of specialized and informative viewpoints for complete coverage. Based on SSD, a hierarchical planner effectively divides the large planning problem into independent sub-problems, enabling parallel planning for each subspace. The carefully designed global and local planning strategies are then incorporated to guarantee both high quality and efficiency in path generation. We conduct extensive benchmark and real-world tests, where FC-Planner computes over 10 times faster compared to state-of-the-art methods with shorter path and more complete coverage. The source code will be made publicly available to benefit the community[3]. Project page: `https://hkust-aerial-robotics.github.io/FC-Planner`.**

I. Introduction

Recently, Unmanned Aerial Vehicles (UAVs) have become increasingly popular to gather information of target scenes for various tasks, such as structural inspection [1, 2] and 3D reconstruction [3], due to their agility and flexibility. To accomplish these tasks, UAVs need to find a shortest path to fully cover 3D scenes, which is typically formulated as a 3D coverage path planning problem [4].

Existing works [1, 2, 5]–[10] commonly address this problem by dividing it into two sub-tasks: 1) identifying a set of viewpoints that cover the target scene, 2) determining the shortest path with complete coverage. However, current solutions for these sub-tasks have limitations that hinder the attainment of satisfactory system simplicity, computation efficiency, and path quality. For the first sub-task, existing methods require pre-processing using extra tools (*e.g.*, Blender [11], CloudCompare [12]) to extract free space for ensuring the safety of viewpoints, which increases system complexity. Moreover, there is a dearth of an efficient strategy that can produce a minimal set of informative viewpoints comprehensively covering the target scene. Existing approaches suffer from a trade-off between viewpoints' number and completeness, *i.e.*, a small number of viewpoints cannot guarantee complete coverage while large amount incurs significant computational costs. On the other hand, the path planner in most frameworks globally optimizes a path fully covering the scene with minimum length, typically formulated as a Combinatorial Optimization Problem (COP) [13]. However, due to the large scale of the planning problem, existing methods frequently struggle to find high-quality solutions promptly in large and complex scenes. Presently, there is a pressing demand for an efficient planning approach in the context of complex 3D scene coverage, which should be capable of decomposing the formidable planning problem into parallelizable sub-problems, ensuring both solution quality and computational efficiency.

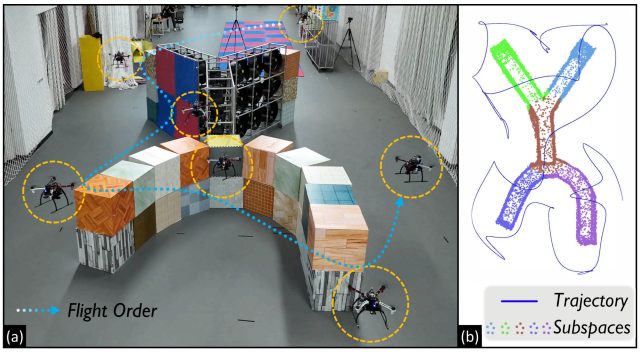

Fig. 1. (a) An aerial coverage test conducted in a challenging scene. (b) Showcase of space decomposition details and coverage trajectory.

To tackle the above limitations, we propose **FC-Planner**, a skeleton-guided planning framework tailored for fast coverage of large and complex 3D scenes. Central to our method is the skeleton-based space decomposition (SSD), which efficiently extracts the skeleton of target scenes. The skeleton provides useful guidance in two aspects. Firstly, it is located inside the target scene, enabling effortless differentiation between internal and external spaces, eliminating the necessity for the pre-processing of free space. Secondly, the skeleton decomposes the complex scene into subspaces characterized by simple geometry. Then, viewpoint sampling rays originated from the skeleton of each subspace are employed to generate specialized viewpoints, thus avoiding redundant sampling of viewpoints and mismatches between viewpoints and their corresponding subspaces. Further, a gravitation-like method is designed to screen out a minimal set of informative viewpoints. Leveraging the straightforward geometry of each subspace and its dedicated set of viewpoints, our approach effectively divides the entire coverage path planning problem into parallelizable sub-problems. This, in conjunction with

[1]School of Artificial Intelligence, Sun Yat-Sen University, Zhuhai, China.
[2]Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong, China.
{cfengag, hlied, xchencq, eeshaojie}@ust.hk, zagerzhang@gmail.com, zhouby23@mail.sysu.edu.cn
† **Corresponding Author**
[3]https://github.com/HKUST-Aerial-Robotics/FC-Planner

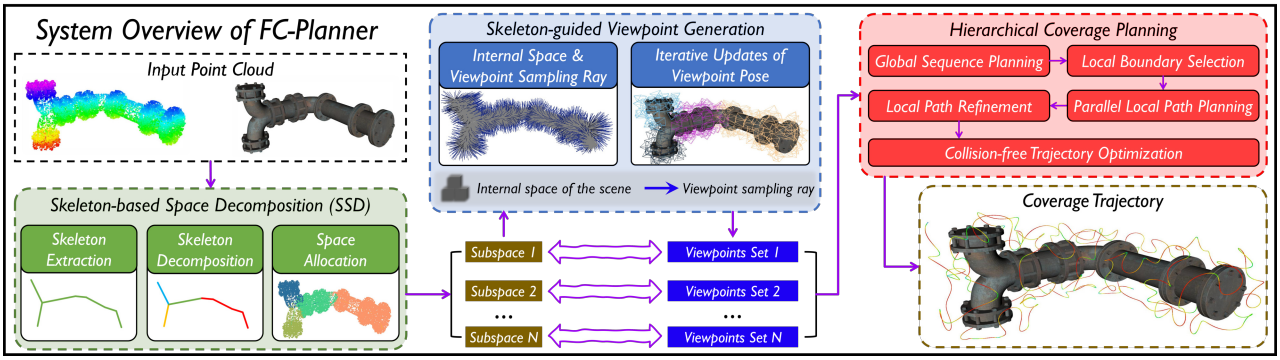

Fig. 2. The overview of the proposed skeleton-guided planning framework for fast aerial coverage in complex 3D scenes.

our carefully designed global and local planning procedures, results in the generation of high-quality coverage paths and high computational efficiency.

We compare our proposed method with state-of-the-art works in simulation. Results show that our method achieves significantly higher computational efficiency (over 10 times faster) and more complete coverage with shorter paths. To further validate **FC-Planner**, we conduct real-world coverage tests in a challenging scene. Both the simulation and real-world experiments demonstrate the superior system simplicity and performance of our method compared to state-of-the-art ones. The contributions of this paper are as follows:

1) An SSD that efficiently extracts the skeleton of complex 3D scenes and guides the coverage planning.

2) An efficient and specialized viewpoint sampling guided by the skeleton, and a gravitation-like model that efficiently screens out the minimal set of informative viewpoints.

3) A hierarchical coverage planner, which divides the entire planning problem into parallelizable sub-problems utilizing SSD. Along with the carefully designed global and local planning modules, it ensures high path quality and computational efficiency.

4) Extensive experiments validate the proposed method. The source code of our implementation will be made public.

II. Related Work

The coverage path planning problem (CPP) in 3D scenes has been an active research topic for UAVs with a wide range of practical applications. Existing methods typically adopt a sampling-based framework to solve such CPP in two steps, namely viewpoint generation and path planning.

Viewpoint Generation: It refers to the process of obtaining a set of viewpoints to fulfill the coverage need of a target scene. Existing methods required pre-processing the scene to identify free space, from which safe and valid viewpoints are then sampled. Some of these methods [1, 5]–[8, 10] utilized triangles of the mesh to sample viewpoints. Alternatively, other works [2, 9, 14] chose the form of point clouds. Our approach falls into the latter category. However, both types often sampled a large number of viewpoint candidates and then iteratively selected a subset of viewpoints to achieve complete coverage. This strategy resulted in vast time consumption for selection and made it challenging to find the fewest viewpoints needed.

In contrast, our method avoids pre-processing by utilizing the skeleton to determine free space and then samples viewpoints around the scene. To efficiently find a minimal set of viewpoints for full coverage, we develop an efficient strategy that iteratively updates the poses of viewpoints using a gravitation-like model without time-consuming selection.

Path Planning: The objective of this step is to plan the shortest collision-free path while comprehensively covering the scene. Previous approaches [1, 5]–[10] formulated this problem as some concrete forms of the COP, *e.g.*, Traveling Salesman Problem (TSP) [15], Submodular Orienteering Problem (SOP) [16], etc. As such problems are known to be NP-hard, the computation time increases drastically with the complexity of the scene. To address this issue, HCPP [2] proposed an approach that divides the scene using an Octree. Each leaf in the Octree contains a maximum of N viewpoints, defining a subspace. Then, the planning problem is transformed into solving local TSPs for each subspace. However, having viewpoints within a single subspace does not guarantee complete coverage of that subspace. Additional viewpoints from other subspaces may be required to achieve full coverage. Thus, the covered area for each subspace cannot be pre-allocated, and the covered goals for the next subspace can only be updated once the planning for the previous subspace is completed. This poses a challenge in achieving parallel planning for different subspaces.

Instead, our SSD decomposes the scene into several simple subspaces guided by the skeleton. This ensures that viewpoints within each subspace avoid mismatches with this subspace. As a result, parallel path generation for each subspace becomes possible, leading to significant savings in computation time.

III. System Overview

The proposed framework takes the point cloud of the target scene as input, which is widely adopted in various applications. As shown in Fig.2, it consists of SSD (Sect.IV), efficient viewpoint generation (Sect.V), and hierarchical coverage planner (Sect.VI). SSD extracts the skeleton of the point cloud and decomposes the cloud into subspaces with simple geometry (Sect.IV). Guided by the skeleton, we determine the internal space and viewpoint sampling rays to sample safe initial viewpoints without pre-processing. Our method then minimizes the number of viewpoints by updating their poses using a gravitation-like model (Sect.V).

Afterwards, the hierarchical planner finds a global visiting sequence of subspaces, optimizes local paths for each subspace in parallel, refines the junction parts between local paths, and generates a trajectory to realize complete coverage (Sect.IV).

IV. Skeleton-based Space Decomposition

In this section, we present our skeleton-based space decomposition in three steps. 1) Compute the skeleton composed of generalized rotational symmetry axis (ROSA) points [17] given the point cloud of the target scene (Sect.IV-A). 2) Decompose the skeleton into several simple shapes, termed branches (Sect.IV-B). 3) Allocate the point cloud into each branch, forming the corresponding subspace (Sect.IV-C).

A. Skeleton Extraction

To begin, we obtain the oriented point cloud $\mathcal{P}_O$ from the input point cloud by normal estimation [18]. Inspired by [17], we represent the skeleton using a set of ROSA points, each of which consists of a position and an orientation denoted as r=($\mathbf{x}, \mathbf{v}$). To accelerate the calculation process, we downsample $\mathcal{P}_O$ as $\mathcal{P}_D$ and compute its ROSA point r_p=($\mathbf{x}_p, \mathbf{v}_p$) of each point $p \in \mathcal{P}_D$. For each p, its corresponding ROSA point operates on the p's neighborhood $\mathcal{N}_p$ within the plane constructed by $\mathbf{v}_\mathrm{p}$ and p. ROSA point exhibits high rotational symmetry about $\mathcal{N}_p$. This property necessitates that the orientation $\mathbf{v}_\mathrm{p}$ minimizes the variance of the angles between $\mathbf{v}_p$ and normals in $\mathcal{N}_p$, while the position $\mathbf{x}_\mathrm{p}$ minimizes the sum of squared distances to the line extensions of normals in $\mathcal{N}_p$. Firstly, given an initial orientation $\mathbf{v}_p^0$, we iteratively optimize $\mathbf{v}_p$ by solving the below quadratic programming problem:

$$\mathbf{v}_p^{i+1} = \underset{\mathbf{v}_p \in \mathbb{R}^3, ||\mathbf{v}_p||_2 = 1}{\arg\min} {\mathbf{v}_p^i}^\mathsf{T} \Sigma_p^i \mathbf{v}_p^i, \tag{1}$$

where $\mathbf{v}_p^i$ denotes the orientation at the i-th iteration. Σ_p^i is the covariance matrix of all point normals in p's neighborhood $\mathcal{N}_p^i$. After finding orientation of ROSA point, we calculate $\mathbf{x}_p$ using the following procedure:

$$\underset{\mathbf{x}_p \in \mathbb{R}^3}{\arg\min} \sum_{p_k \in \mathcal{N}_p} ||(\mathbf{x}_p - p_k) \times \mathbf{n}(p_k)||_2^2, \tag{2}$$

where $\mathbf{n}(p_k)$ denotes point normal of $p_k \in \mathcal{N}_p$. Lastly, we apply 1D moving least square [19] on all ROSA points to generate the skeleton. The skeleton is stored in an undirected graph $\mathcal{G} = (V_s, E_s)$, where vertices V_s contain all ROSA points while edges E_s represent their connections.

B. Skeleton Decomposition

As outlined in Algo.1, we decompose the extracted skeleton into several branches, each of which comprises a set of edges in $\mathcal{G}$. These branches are all contained in the set of branches denoted by $\mathcal{B}$. In line 2, we identify joints $\mathcal{J}$ on $\mathcal{G}$, which are defined as vertices with their degree greater than two. Due to simple geometry requirements, a branch typically starts with a joint and ends with a joint or a leaf vertex (degree equals one). To obtain branches satisfying the above needs, we develop a depth-first search (DFS)-like procedure [20] in lines 3-14. Further, to ensure that each branch is as simple and easy to cover as possible, we require that edges within one branch should have similar directions. Thus, in lines 16-26 we decompose the branches with large direction changes into simpler ones.

Algorithm 1: Skeleton Decomposition

Input: Skeleton graph $\mathcal{G} = (V_s, E_s)$, Angle variance maximum δ
Output: Branches set $\mathcal{B}$
1 **Define:** Joints set $\mathcal{J}$, Branch $\mathcal{T}_b$, Current vertex v_c, Next vertex v_n
2 $\mathcal{J} \leftarrow \mathrm{FindJoint}(V_s)$;
3 **for** $v_j \in \mathcal{J}$ **do**
4 **for** $v_{adj} \in v_j$.AdjacentVertices **do**
5 $\mathcal{T}_b$.clear(); $v_c = v_j$, $v_n = v_{adj}$;
6 $\mathcal{T}_b$.AddEdge($v_c v_n$);
7 **while** $deg(v_n) = 2$ **do**
8 $\mathcal{Q} \leftarrow v_n$.AdjacentVertices.remove(v_c);
9 $v_c = v_n$, $v_n = \mathcal{Q}$.first();
10 $\mathcal{T}_b$.AddEdge($v_c v_n$);
11 **end**
12 $\mathcal{B}$.Add($\mathcal{T}_b$);
13 **end**
14 **end**
15 **Define:** Temporary branches set $\mathcal{B}_T$, Reference direction angle σ
16 **for** $b \in \mathcal{B}$ **do**
17 $\mathcal{T}_b$.clear(); $\mathcal{T}_b$.AddEdge(b[0]); $\sigma = \mathrm{CalculateAngle}(b[0])$;
18 **for** $i \in [1, b.size())$ **do**
19 **if** $||\mathrm{CalculateAngle}(b[i]) - \sigma|| < \delta$ **then**
20 $\mathcal{T}_b$.AddEdge(b[i]);
21 **else**
22 $\mathcal{B}_T$.Add($\mathcal{T}_b$); $\mathcal{T}_b$.clear();
23 $\mathcal{T}_b$.AddEdge(b[i]); $\sigma = \mathrm{CalculateAngle}(b[i])$;
24 **end**
25 **end**
26 **end**
27 $\mathcal{B} = \mathcal{B}_T$.

C. Space Allocation

Given the decomposed skeleton, we aim to allocate the points in $\mathcal{P}_O$ into multiple subspaces, each of which corresponds to a branch in $\mathcal{B}$. We discretize edges in those branches into several oriented points, whose orientation is the direction of the edge it is on. The planes determined by these oriented points are denoted as Γ. Then, for each plane, we query points within this plane from $\mathcal{P}_O$. If one point $p \in \mathcal{P}_O$ is located on more than one plane, p will be allocated to the plane whose oriented point is nearest to it. After that, the points queried by planes (named allocated points $\mathcal{A}$) from the same branch are included in a subspace. All subspaces are contained in $\mathcal{S} = \{s_1, ..., s_N\}$.

V. Skeleton-guided Viewpoint Generation

This section presents the efficient generation of viewpoints for full coverage. The skeleton allows for the identification of internal space that ensures the safety of viewpoints without pre-processing. Additionally, the skeleton spawns a set of viewpoint sampling rays, guiding the production of viewpoints (Sect.V-A). To efficiently find a minimal set of viewpoints, we propose a gravitation-like model that iteratively adjusts the pose of viewpoints and removes redundant ones, screening out a more representative subset (Sect.V-C). This process is further accelerated by bidirectional ray casting (BiRC) that allows for faster visibility checks (Sect.V-B).

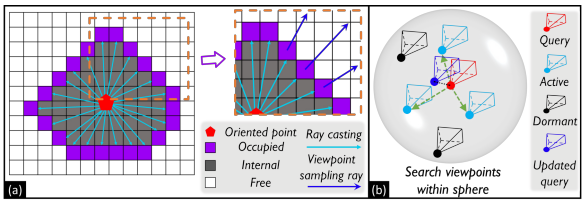

Fig. 3. (a) Generation of internal space and viewpoint sampling rays (Sect.V-A). (b) Gravitation-like model used to update the pose of query viewpoint (Sect.V-C).

A. Internal Space and Viewpoint Sampling Ray

The internal space is represented using a volumetric map whose all voxels are initialized as free. The voxels that contain $\mathcal{P}_O$ are updated as occupied. For each plane $\gamma_i \in \Gamma$ (Sect.IV-C), we perform ray casting from its oriented point o_i (located on the skeleton) to each of its allocated points $\mathcal{A}_i$. Fig.3(a) shows that voxels along the rays before crossing the occupied ones are labeled as internal. The rays continue to extend into the free space, and we define the parts of these rays starting from the occupied voxels as viewpoint sampling rays. Safe and informative viewpoints are sampled along them, reducing the number of redundantly sampled viewpoints. These specialized viewpoints also provide better coverage since the skeleton guides them all pointing towards the surface. Moreover, the viewpoint sampling rays that cast from o_i belong to the subspace that corresponds to o_i.

B. Bidirectional Ray Casting

Ray casting is widely used for the visibility check of viewpoints. However, this approach traverses voxels unidirectionally, commencing from one end of the ray. It may not efficiently identify occlusions when an occupied voxel is located near the other end of the ray. To address this issue, we implement bidirectional ray casting (BiRC), wherein voxels are traversed from both ends toward the midpoint of the ray. This approach stops earlier and eliminates redundant checks in the face of the aforementioned situation.

C. Iterative Updates of Viewpoint Pose

Our method employs 5-DoF viewpoints, denoted as $\mathbf{vp} = [\mathbf{p}, \theta, \phi, id]$, where $\mathbf{p}$ is the 3D position of the sensor, θ, ϕ respectively represent pitch and yaw angles. The parameter id is the subspace to which the viewpoint belongs. The start and direction of viewpoint sampling ray r_{vs} is respectively described as $\mathbf{sr} = [x_{sr}, y_{sr}, z_{sr}]$ and $\mathbf{dr} = [nx_{dr}, ny_{dr}, nz_{dr}]$. We sample one viewpoint along each r_{vs} at distance D away, the id of which refers to the subspace r_{vs} belongs to. The position, pitch, and yaw of the sampled viewpoint are determined as follows:

$$\begin{aligned} &\mathbf{p} = \mathbf{sr} + D * \mathbf{dr}, \theta = \arcsin(-nz_{dr}/||\mathbf{dr}||_2), \\ &\phi = \arctan(-ny_{dr}/-nx_{dr}). \end{aligned} \tag{3}$$

These sampled viewpoints are named as initial viewpoints $\mathcal{VP}_{ini}$. The first step is to determine voxels covered by each viewpoint in $\mathcal{VP}_{ini}$ using the proposed BiRC. Next, to reduce the number of viewpoints, we assign each voxel to the viewpoint that covers the most voxels if it is observed by more than two viewpoints. We then remove the viewpoints that are not assigned any voxels from $\mathcal{VP}_{ini}$. Afterwards,

Algorithm 2: Local Path Refinement

Input: Coverage path $\mathcal{P}_\mathcal{C}$, branches set $\mathcal{B}$, junction radius r_{jc}
Output: Refined coverage path $\mathcal{P}_\mathcal{R}$

$\mathcal{Z} \leftarrow \text{FindJunctionVertex}(\mathcal{B})$; $T_\mathcal{C} \leftarrow \text{BuildKdTree}(\mathcal{P}_\mathcal{C})$;
for $z \in \mathcal{Z}$ **do**
  $V_{jc} \leftarrow T_\mathcal{C}.\text{RadiusSearch}(z, r_{jc})$;
  for $i \in [1, K]$ **do**
    $v_1 \leftarrow \text{RandomPick}(V_{jc})$; $V_{temp} \leftarrow V_{jc}.\text{remove}(v_1)$;
    $v_2 \leftarrow \text{RandomNeighbor}(v_1)$;
    if $v_2 == v_1.Suc$ **then**
      $V_{temp} \leftarrow V_{temp}.\text{remove}(v_2 \,\&\, v_2.Suc)$;
    else
      $V_{temp} \leftarrow V_{temp}.\text{remove}(v_2 \,\&\, v_2.Pred)$;
    end
    $v_3 \leftarrow \text{RandomPick}(V_{temp})$;
    $\mathcal{P}_\mathcal{R} \leftarrow \text{Make2Opt}(v_1, v_2, v_3)$;
  end
end

we construct a kd tree [21] T_{ini} for $\mathcal{VP}_{ini}$ and define a binary state (active or dormant) for each viewpoint, initially setting all viewpoints as active. To ensure that each viewpoint covers as many voxels as possible, we merge the viewpoints with fewer voxels into the viewpoints with more voxels using a gravitation-like model. Specifically, commencing with the viewpoint that covers the maximum number of voxels and proceeding to the one that covers the minimum, each viewpoint (denoted as $\mathbf{vp}_q$) queries its neighborhood $\mathcal{VP}_q$ from T_{ini} using a radius search. The radius r_q is determined by maximal visible distance d_v and the Field of View (FoV) $[f_h, f_w]$:

$$r_q = d_v * \tan(\min(f_h, f_w)/2). \tag{4}$$

Fig.3(b) illustrates how we update the pose of the query viewpoint $\mathbf{vp}_q$ using all active viewpoints $\mathcal{VP}_a$ in $\mathcal{VP}_q$ through a gravitation-like model:

$$\overline{\mathbf{p}_q} = \mathbf{p}_q + \sum_{\mathbf{vp}_a \in \mathcal{VP}_a} \frac{c_a}{c_q}(\mathbf{p}_a - \mathbf{p}_q), \text{ s.t. } c_a < c_q, \tag{5}$$

where c_q and c_a denote the number of voxels covered by $\mathbf{vp}_q$ and each viewpoint in $\mathcal{VP}_a$, respectively. The updated position of $\mathbf{vp}_q$ is represented as $\overline{\mathbf{p}_q}$. Similarly, we obtain the updated pitch $\overline{\theta_q}$ and yaw $\overline{\phi_q}$. They are further adjusted to distribute voxels covered by $\mathbf{vp}_q$ around the center of FoV. The subspace to which $\mathbf{vp}_q$ belongs is the same as that of the nearest viewpoint to $\mathbf{vp}_q$ in $\mathcal{VP}_{ini}$. Next, we set the state of the viewpoints used to update $\mathbf{vp}_q$ in $\mathcal{VP}_a$ as dormant. After that, we identify uncovered voxels by performing the first step for all active viewpoints and then sample new viewpoints to cover them, denoted as $\mathcal{VP}_{unc}$. Lastly, we repeat the above steps for $\mathcal{VP}_{unc}$ achieving complete coverage. After such iterative updates for the poses of viewpoints, all remaining active viewpoints are assigned to each subspace in $\mathcal{S}$ according to their id, with each subset of viewpoints represented as $\mathcal{V} = \{\mathcal{VP}_1, ..., \mathcal{VP}_N\}$.

VI. Hierarchical Coverage Planning

Given the above viewpoints, this stage endeavors to find the shortest collision-free path traversing them. Supported by SSD, we decompose large planning problem into several independent and parallelizable sub-problems in five steps.

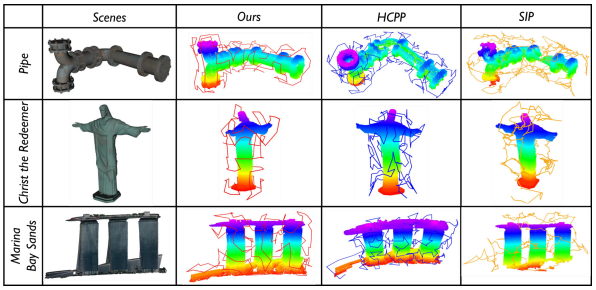

Fig. 4. Comparisons on coverage situation and generated path of the proposed method, SIP [1], and HCPP [2] in three complex scenarios.

Global Sequence Planning. It seeks to solve an optimal tour visiting all subspaces starting from UAV's current pose, which is defined as an Asymmetric Traveling Salesman Problem (ATSP) [22]. We first calculate the centroid of viewpoints in each subspace and then construct a Euclidean distance matrix $\mathcal{M}_G$, between these centroids and UAV's current pose. Lin-Kernighan-Helsgaun (LKH) solver [23] takes $\mathcal{M}_G$ as input to obtain global sequence $[g_1, ..., g_N]$.

Local Boundary Selection. We expect to select suitable start and end viewpoints for each subspace, ensuring that the results of local planning are compatible with the global sequence. According to global sequence, we sort all centroids as $Seq_C = [\mathbf{k}_0, ..., \mathbf{k}_N]$, where $\mathbf{k}_0$ is UAV's current pose and others are sorted centroids. Based on Seq_C, the start $\mathbf{vp}^{g_i}_{start}$ and end $\mathbf{vp}^{g_i}_{end}$ of i-th visited subspace are obtained as:

$$\begin{aligned}\mathbf{vp}^{g_i}_{start} &= \mathop{\arg\min}_{\mathbf{vp}_{g_i} \in \mathcal{VP}_{g_i}} ||\mathbf{p}_{g_i} - \mathbf{k}_{i-1}||_2^2 + ||\mathbf{p}_{g_i} - \mathbf{k}_i||_2^2, \\ \mathbf{vp}^{g_i}_{end} &= \mathop{\arg\min}_{\mathbf{vp}_{g_i} \in \mathcal{VP}_{g_i}} ||\mathbf{p}_{g_i} - \mathbf{k}_i||_2^2 + ||\mathbf{p}_{g_i} - \mathbf{k}_{i+1}||_2^2.\end{aligned} \tag{6}$$

In particular, the last visited subspace will not have an end viewpoint since there is no subsequent subspace.

Parallel Local Path Planning. Once determining boundary viewpoints, we can individually plan local paths for each subspace. We formulate this problem as a variant of TSP that adds a constraint of given start and end. Suppose a subspace has R viewpoints, cost matrix $\mathcal{M}_L$ for this TSP follows:

$$\mathcal{M}_L(i,j) = \begin{cases} 0, & i == j \; or \; j = 0 \\ +\infty, & i = R-1 \; and \; j \in \{1, ..., R-2\} \\ c_{\mathbf{vp}_i, \mathbf{vp}_j}, & others \end{cases},$$

$$c_{\mathbf{vp}_i, \mathbf{vp}_j} = \max(\max(\frac{\mathrm{L}(\mathbf{p}_i, \mathbf{p}_j)}{\mathrm{v}_{max}}, \frac{\mathrm{Ang}(\theta_i, \theta_j)}{\omega_{max}}), \frac{\mathrm{Ang}(\phi_i, \phi_j)}{\omega_{max}}),$$
$$\mathrm{Ang}(\mathrm{a}_1, \mathrm{a}_2) = \min(|\mathrm{a}_1 - \mathrm{a}_2|, 2\pi - |\mathrm{a}_1 - \mathrm{a}_2|), \tag{7}$$

where $\mathrm{L}(\cdot)$ is the path length between v_i and v_j searched by A^* algorithm to warrant collision-free path, v_{max} and ω_{max} are the limits of velocity and angular rate of pitch and yaw. For the last visited subspace, all $+\infty$ entries in $\mathcal{M}_L$ are set as $c_{\mathbf{vp}_i, \mathbf{vp}_j}$. For each $\mathcal{VP}_i \in \mathcal{V}$, we use multi-thread programming to parallelly produce local path that covers its corresponding subspace. Let there be a total of $m \in \mathbb{R}^+$ viewpoints and the maximal number of viewpoints in all sub-problems is $e \in \mathbb{R}^+$, $m \gg e$. Since the time complexity of solving a TSP using the LKH solver is $\mathcal{O}(n^{2.2})$ [24], solving for the TSP of all viewpoints without problem decomposition takes $\mathcal{O}(m^{2.2})$ time. Thanks to our parallel planning, the time complexity of solving for all sub-problems is reduced to $\mathcal{O}(e^{2.2})$. Hence, our method effectively lowers the time complexity and improves computational efficiency. We combine all local paths in the order of global sequence to obtain the entire coverage path $\mathcal{P}_\mathcal{C}$.

TABLE I

COVERAGE PLANNING COMPARISONS IN THREE SCENARIOS.

Scenario	Method	Pre-process	Comp. Time (ms)	Viewpoint Number	Path Length (m)	Exec. Time (s)	Coverage Rate (%)
Pipe	SIP [1]	✓	235012.1	3078	2754.1	3938.8	69.2
	HCPP [2]	✓	16934.2	446	1874.3	1670.9	87.3
	Ours	✗	**1614.8**	**210**	**1440.4**	**1033.4**	**97.5**
Christ the Redeemer	SIP [1]	✓	62110.9	787	700.5	1104.8	85.9
	HCPP [2]	✓	5485.7	228	635.0	642.3	97.4
	Ours	✗	**506.7**	**111**	**530.4**	**386.9**	**99.7**
Marina Bay Sands	SIP [1]	✓	131454.8	1770	1635.5	2789.6	68.5
	HCPP [2]	✓	12639.4	369	1698.9	1613.9	89.3
	Ours	✗	**954.3**	**185**	**1396.2**	**1178.5**	**94.0**

Local Path Refinement. Since the coverage path in the previous step is optimized within each subspace, it cannot guarantee global optimality thus it may produce unnecessary detours. We have observed that these detours typically occur near the junctions between local paths. Thus, we propose a local refinement strategy to further improve the entire coverage path on junctions, as depicted in Algo.2. A junction vertex is defined as a vertex where two branches intersect in line 1. Lines 2-16 show the iterative refinement procedure using a local search in two steps, which continues K iterations. It first chooses the parts from the whole coverage path that are near the junctions. Then, each iteration randomly selects three viewpoints from these parts and swaps them using 2-opt [25] if this swap can shorten the coverage path.

Collision-free Trajectory Optimization. It aims to convert refined coverage path $\mathcal{P}_\mathcal{R} = \{v^0_\mathcal{R}, ..., v^M_\mathcal{R}\}$ to a smooth, dynamically feasible, safe, and minimum-time trajectory passing through all viewpoints. Specifically, we divide $\mathcal{P}_\mathcal{R}$ into M trajectory pieces and then generate 3D Safe Flight Corridors (SFCs), where each convex flight corridor is assigned with a consecutive set of trajectory pieces. For position trajectory, SFCs are used to ensure safety:

$$tp_i(t) \in CP(i), \forall t \in [0, T_i], \forall 1 \le i \le M, \tag{8}$$

where tp_i is the i-th trajectory piece with its duration T_i and $CP(i)$ denotes the assigned convex flight corridor for tp_i in SFCs. To avoid visual blur caused by aggressive flight as well as ensuring dynamic feasibility, the velocity, acceleration and jerk limits are also considered:

$$\begin{aligned} &||\dot{tp_i}(t)||_2^2 \le \mathrm{v}^2_{max}, \forall t \in [0, T_i], \forall 1 \le i \le M, \\ &||\ddot{tp_i}(t)||_2^2 \le \mathrm{a}^2_{max}, \forall t \in [0, T_i], \forall 1 \le i \le M, \\ &||\dddot{tp_i}(t)||_2^2 \le \mathrm{j}^2_{max}, \forall t \in [0, T_i], \forall 1 \le i \le M, \\ &\text{s.t. } tp_i(0) = \mathbf{p}^{i-1}_\mathcal{R}, tp_i(T_i) = \mathbf{p}^i_\mathcal{R}, \end{aligned} \tag{9}$$

where v_{max}, a_{max}, and j_{max} are dynamic limits. We also generate trajectories for pitch and yaw angles, where the maximum angular velocity is limited similarly to Eq.9.

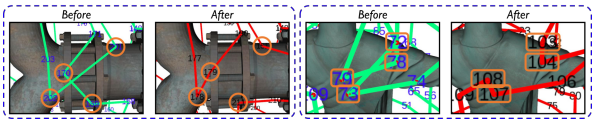

Fig. 5. Comparisons of results before and after using local path refinement.

We utilize MINCO [26] to optimize the time allocation of coverage trajectory while satisfying the above constraints.

VII. Experiments

A. Implementation Details

During the skeleton extraction step, we normalize the input point cloud within a unit sphere, which enhances its robustness to variations in scene scale We set $\delta = 45°$ in Algo.1, the number of iterations in local path refinement $K = 10000$ since each iteration only consumes around 0.01ms. In all experiments, we choose a 2-axis gimbal camera as the sensor equipped with the UAV. A geometric controller [27] is used for tracking control of $(\mathbf{p}, \theta, \phi)$ trajectory. All modules run on an Intel Core i9-13900K CPU.

B. Benchmark Comparisons and Analysis

To validate our proposed framework, we benchmark it in simulation including three large and complex scenes, *Pipe* (**73**×**66**×**41** m^3), *Christ the Redeemer* (**10**×**36**×**38** m^3), and *Marina Bay Sands* (**100**×**25**×**49** m^3). Two state-of-the-art methods are compared: SIP [1] and HCPP [2]. There is no open source code for HCPP [2], so we use our implementation. In simulations, the pitch angle of gimbal is limited within $[-90°, 70°]$ with camera FoV $[75°, 55°]$. We set dynamic limits as $v_{max} = 2.0$ m/s, $\omega_{max} = 1.0$ rad/s, $a_{max} = 1.0$ m/s^2, and $j_{max} = 0.5$ m/s^3 for all approaches. The average comparison results are reported in Table.I, where Exec. Time means trajectory execution time. We also visualize the comparisons in Fig.4.

TABLE II

Ablation Study on The Proposed Planning Modules.

	Pipe			*Christ the Redeemer*			*Marina Bay Sands*		
	NR	Ours	**GO**	**NR**	Ours	**GO**	**NR**	Ours	**GO**
Comp. Time (ms)	**1461.5**	1614.8	7126.7	**397.2**	506.7	899.5	**843.2**	954.3	4901.7
Path Length (m)	1480.1	1440.4	**1381.9**	581.2	530.4	**493.4**	1561.9	1396.2	**1164.8**

From the evaluation, our method significantly outperforms others in computation efficiency, coverage completeness, and path quality. Our framework achieves maximal coverage with minimal viewpoints due to the proposed skeleton-guided viewpoint generation. Thanks to our effective SSD and parallel planning, our approach runs over 10x faster than existing methods. For path quality, we generate a shorter coverage path and trajectory realizing less execution time. Additionally, such advantages become more pronounced as the complexity and scale of the scene increase.

TABLE III

Computation Time of Each Module.

	SSD	Subspaces	Viewpoint Generation	Hierarchical Planning	
				Safe Path Search	Planning
Pipe	∼80ms	4	∼160ms	∼1000ms	∼200ms
Christ the Redeemer	∼35ms	4	∼80ms	∼210ms	∼180ms
Marina Bay Sands	∼300ms	10	∼150ms	∼300ms	∼200ms

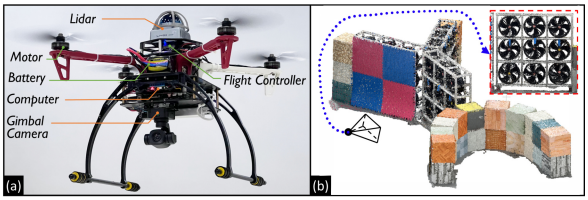

Fig. 6. (a) The quadrotor with gimbal camera platform used for real-world coverage test. (b) The 3D reconstruction results after aerial coverage.

We perform ablation studies to demonstrate the effectiveness of local path refinement and hierarchical planning. We denote our framework without local refinement as **NR**, while **GO** replaces hierarchical planning with global optimization of the path for all viewpoints together. Table.II shows the contributions of local path refinement and hierarchical planning to path quality and computation efficiency. Our method shortens the path with minimal additional computation compared to **NR**. Fig.5 presents the effectiveness of local refinement. Compared to **GO**, our approach markedly reduces computation time with minimal loss in path quality. Moreover, we provide the runtime of each module and space decomposition details in Table.III. Table.III also provides runtime of each module and space decomposition details. Moreover, detailed performance information for each module can be found on our project page.

C. Real-world Test

To further verify the proposed framework, we conduct a real-world coverage test in a challenging scene as shown in Fig.1(a), the size of which is **10**×**5**×**2.5** m^3. For this test, we design a customized quadrotor as presented in Fig.6(a). The gimbal rotates between $[-90°, 20°]$ and the camera FoV is $[60°, 80°]$. The dynamic limits are set as $v_{max} = 0.7$ m/s, $\omega_{max} = 1.0$ rad/s, $a_{max} = 0.5$ m/s^2, and $j_{max} = 0.5$ m/s^3. We first collect the point cloud of the scene using Lidar. Then, our framework produces the coverage trajectory as shown in Fig.1(b), where our method reasonably decomposes the scene into five subspaces for parallel planning. Afterwards, the quadrotor captures images during executing the trajectory. Lastly, we use these images to reconstruct the scene and the results shown in Fig.6(b) demonstrate our method indeed achieves complete coverage. More details about this test can be found in our video.

VIII. Conclusions

In this study, we present a skeleton-guided planning framework designed specifically for fast coverage of large and complex 3D scenes. The proposed SSD efficiently extracts the scene's skeleton, enabling reasonable decomposition of the scene into simple subspaces and guiding efficient generation of specialized and informative viewpoints without preprocessing. Based on SSD, the hierarchical coverage planner divides the entire planning problem into parallelizable subproblems, combining the devised global planning and local refinement approaches to efficiently produce the high-quality coverage path. The extensive experiments demonstrate the efficiency and superiority of our framework. In the future, we plan to extend our framework to multi-UAV system for cooperative coverage in larger and unknown 3D scenes.

REFERENCES

[1] A. Bircher, K. Alexis, M. Burri, P. Oettershagen, S. Omari, T. Mantel, and R. Siegwart, "Structural inspection path planning via iterative viewpoint resampling with application to aerial robotics," in *2015 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2015, pp. 6423–6430.

[2] C. Cao, J. Zhang, M. Travers, and H. Choset, "Hierarchical coverage path planning in complex 3d environments," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 3206–3212.

[3] C. Feng, H. Li, F. Gao, B. Zhou, and S. Shen, "Predrecon: A prediction-boosted planning framework for fast and high-quality autonomous aerial reconstruction," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 1207–1213.

[4] E. Galceran and M. Carreras, "A survey on coverage path planning for robotics," *Robotics and Autonomous systems*, vol. 61, no. 12, pp. 1258–1276, 2013.

[5] W. Jing, J. Polden, W. Lin, and K. Shimada, "Sampling-based view planning for 3d visual coverage task with unmanned aerial vehicle," in *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2016, pp. 1808–1815.

[6] R. Almadhoun, T. Taha, D. Gan, J. Dias, Y. Zweiri, and L. Seneviratne, "Coverage path planning with adaptive viewpoint sampling to construct 3d models of complex structures for the purpose of inspection," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 7047–7054.

[7] W. Jing, D. Deng, Z. Xiao, Y. Liu, and K. Shimada, "Coverage path planning using path primitive sampling and primitive coverage graph for visual inspection," in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2019, pp. 1472–1479.

[8] H. W. Tong, B. Li, H. Huang, and C.-y. Wen, "Uav path planning for complete structural inspection using mixed viewpoint generation," in *2022 17th International Conference on Control, Automation, Robotics and Vision (ICARCV)*. IEEE, 2022, pp. 727–732.

[9] X. Huang, Y. Liu, L. Huang, S. Stikbakke, and E. Onstein, "Bim-supported drone path planning for building exterior surface inspection," *Computers in Industry*, vol. 153, p. 104019, 2023.

[10] M. Roberts, D. Dey, A. Truong, S. Sinha, S. Shah, A. Kapoor, P. Hanrahan, and N. Joshi, "Submodular trajectory optimization for aerial 3d scanning," in *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 5324–5333.

[11] "Blender," https://www.blender.org/.

[12] "Cloudcompare," https://www.cloudcompare.org/.

[13] C. H. Papadimitriou and K. Steiglitz, *Combinatorial optimization: algorithms and complexity*. Courier Corporation, 1998.

[14] R. M. Claro, M. I. Pereira, F. S. Neves, and A. M. Pinto, "Energy efficient path planning for 3d aerial inspections," *IEEE Access*, vol. 11, pp. 32 152–32 166, 2023.

[15] G. Dantzig, R. Fulkerson, and S. Johnson, "Solution of a large-scale traveling-salesman problem," *Journal of the operations research society of America*, vol. 2, no. 4, pp. 393–410, 1954.

[16] C. Chekuri and M. Pal, "A recursive greedy algorithm for walks in directed graphs," in *46th annual IEEE symposium on foundations of computer science (FOCS'05)*. IEEE, 2005, pp. 245–253.

[17] A. Tagliasacchi, H. Zhang, and D. Cohen-Or, "Curve skeleton extraction from incomplete point cloud," in *ACM SIGGRAPH 2009 papers*, 2009, pp. 1–9.

[18] R. B. Rusu and S. Cousins, "3d is here: Point cloud library (pcl)," in *2011 IEEE international conference on robotics and automation*. IEEE, 2011, pp. 1–4.

[19] I.-K. Lee, "Curve reconstruction from unorganized points," *Computer aided geometric design*, vol. 17, no. 2, pp. 161–177, 2000.

[20] R. Tarjan, "Depth-first search and linear graph algorithms," *SIAM journal on computing*, vol. 1, no. 2, pp. 146–160, 1972.

[21] K. Zhou, Q. Hou, R. Wang, and B. Guo, "Real-time kd-tree construction on graphics hardware," *ACM Transactions on Graphics (TOG)*, vol. 27, no. 5, pp. 1–11, 2008.

[22] Z. Meng, H. Qin, Z. Chen, X. Chen, H. Sun, F. Lin, and M. H. Ang, "A two-stage optimized next-view planning framework for 3-d unknown environment exploration, and structural reconstruction," *IEEE Robotics and Automation Letters*, vol. 2, no. 3, pp. 1680–1687, 2017.

[23] K. Helsgaun, "An effective implementation of the lin–kernighan traveling salesman heuristic," *European journal of operational research*, vol. 126, no. 1, pp. 106–130, 2000.

[24] C. H. Papadimitriou, "The complexity of the lin–kernighan heuristic for the traveling salesman problem," *SIAM Journal on Computing*, vol. 21, no. 3, pp. 450–465, 1992.

[25] S. M. McGovern and S. M. Gupta, "2-opt heuristic for the disassembly line balancing problem," in *Environmentally conscious manufacturing iii*, vol. 5262. SPIE, 2004, pp. 71–84.

[26] Z. Wang, X. Zhou, C. Xu, and F. Gao, "Geometrically constrained trajectory optimization for multicopters," *IEEE Transactions on Robotics*, vol. 38, no. 5, pp. 3259–3278, 2022.

[27] T. Lee, M. Leoky, and N. H. McClamroch, "Geometric tracking control of a quadrotor uav on se (3)," in *Proc. of the IEEE Control and Decision Conf. (CDC)*, Atlanta, GA, Dec. 2010, pp. 5420–5425.